

*notion에서 수식이 깨져 코랩에서 작성했습니다!

Attention Is All You Need

Vaswani et al., NeurIPS 2017

RNN 없이 attention만으로 sequence를 처리해, 성능과 학습 효율을 동시에 개선한 논문.

0. Abstract

- 기존 sequence transduction model
 - RNN/CNN 기반 encoder-decoder
 - 성능 개선을 위해 attention 추가
- 문제: RNN의 sequential computation
 - 앞 token 계산 → 다음 token 계산
 - sequence 내부 병렬화 어려움
- 제안: Transformer — recurrence X, convolution X, attention만 사용
- 결과
 - WMT 2014 EN-DE: **28.4 BLEU**
 - WMT 2014 EN-FR: **41.8 BLEU**
 - 기존 모델 대비 높은 parallelization, 낮은 training cost
- translation 외 English constituency parsing에서도 좋은 성능

⇒ Attention만으로 sequence를 처리해 성능 + 학습 효율 모두 개선

1. Introduction

1.1 기존 Sequence Model: RNN

기존 sequence modeling에는 RNN, LSTM, GRU가 주로 사용되었고, 현재 hidden state는 이전 hidden state에 의존한다.

$$h_t = f(h_{t-1}, x_t)$$

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_n$$

문제

- h_t 계산은 h_{t-1} 계산이 끝나야 시작 가능
- token들을 동시에 계산할 수 없음

- sequence 길이 $\uparrow \rightarrow$ 학습 시간 $\uparrow$
- memory 제한 $\rightarrow$ batch 크기에도 제약

$\Rightarrow$ RNN의 sequential structure 자체가 parallelization을 제한

1.2 Attention의 등장

attention: 현재 위치가 다른 위치 중 어떤 정보를 얼마나 참고할지 결정하는 mechanism

- 두 위치 사이 거리에 관계없이 dependency 계산 가능
- long-range dependency 처리에 유리

기존에는 RNN + Attention 형태로 사용되었다. Attention이 먼 정보를 직접 참고할 수 있어도 backbone인 RNN 구조는 그대로다.

$\Rightarrow$ attention을 추가하는 것만으로는 sequential computation 문제 해결 $\times$

1.3 Transformer의 핵심 아이디어

Attention이 token 간 관계를 직접 계산할 수 있다면, RNN이 꼭 필요한가?



효과

- sequence 내부 병렬 계산 가능
- global dependency 직접 학습
- 빠른 training

$\Rightarrow$ attention을 보조 mechanism이 아닌 sequence modeling의 핵심 연산으로 사용

2. Background

2.1 기존 병렬화 시도

RNN의 sequential computation을 해결하려는 시도로 CNN 기반 sequence model이 등장했다: Extended Neural GPU, ByteNet, ConvS2S.

- CNN은 여러 input position을 병렬 계산할 수 있다
- 반면 멀리 떨어진 위치를 연결하려면 여러 layer가 필요하다

Model	두 위치를 잇는 operation 수
ConvS2S	거리에 따라 linear 증가
ByteNet	거리에 따라 logarithmic 증가

path length $\uparrow \rightarrow$ long-range dependency 학습 어려움 $\uparrow$

$\Rightarrow$ CNN은 병렬화는 가능하지만 먼 token을 직접 연결하지는 못함

2.2 Self-Attention

self-attention: 하나의 sequence 안에서 서로 다른 위치의 관계를 계산해 각 위치의 representation을 만드는 attention

목표

1. 여러 위치를 동시에 처리
2. 멀리 떨어진 위치도 직접 연결

기존에도 reading comprehension, summarization, textual entailment, sentence representation 등에 활용되었으나, 주로 RNN 등 다른 구조와 **함께** 사용되었다. Transformer는 self-attention만으로 encoder/decoder representation을 계산한다.

⇒ 새로운 attention을 만든 것보다 **self-attention**을 전체 **architecture**의 중심으로 사용한 것이 핵심

3. Model Architecture

기본 골격은 기존 sequence-to-sequence와 동일한 encoder-decoder다.

Input → Encoder → Continuous Representation → Decoder → Output

- **Encoder:** input sequence → continuous representation
- **Decoder:** encoder representation + 이전 output → 다음 token 생성 (auto-regressive)
- **차이점:** RNN/CNN 대신 Attention + Feed Forward 사용

3.1 Encoder and Decoder Stacks

Encoder — 동일한 layer 6개 stack

각 layer는 두 개의 sub-layer로 구성된다.

```

Input
↓
Multi-Head Self-Attention
↓
Add & Norm
↓
Feed Forward
↓
Add & Norm
  
```

- **Self-Attention:** 각 input token이 다른 모든 input token 참고
- **FFN:** attention으로 얻은 각 token representation을 다시 변환

Residual Connection + Layer Normalization

모든 sub-layer에 적용된다.

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- **residual connection:** sub-layer의 변환 결과에 기존 입력 x 를 다시 더함
- **layer normalization:** activation을 normalization해 학습 안정화
- residual addition이 성립하려면 모든 sub-layer의 차원이 같아야 한다

$$d_{\text{model}} = 512$$

Decoder — 동일한 layer 6개 stack

Encoder보다 attention layer가 하나 많다.

```

Previous Output
    ↓
Masked Self-Attention
    ↓
Add & Norm
    ↓
Encoder-Decoder Attention
    ↓
Add & Norm
    ↓
Feed Forward
    ↓
Add & Norm
  
```

Masked Attention

다음 token을 예측할 때 미래 정답을 보면 안 되므로, 미래 위치의 attention score를 $-\infty$ 로 설정한다.

$$\text{softmax}(-\infty) \approx 0$$

⇒ 현재 위치 이전 token만 참고 가능 → auto-regressive generation 유지

3.2 Attention

attention: 현재 token이 다른 token 중 누구를 얼마나 참고할지 결정하고, 필요한 정보를 가져오는 연산

기호	이름	의미
Q	Query	현재 token이 어떤 정보를 찾고 있는가
K	Key	각 token이 어떤 정보와 관련 있는가
V	Value	각 token이 실제로 전달할 정보

현재 token의 Query

↓

다른 token의 Key와 비교 → 관련성 계산

↓

Attention Weight

↓

해당 token의 Value 가져오기

↓

현재 token representation 갱신

- Q - K : 누구를 볼지 결정
- V : 실제 정보 전달

3.2.1 Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

1. Query-Key Similarity — QK^T

Query와 모든 Key의 dot product를 계산한다. 값이 크면 관련성이 높고, 작으면 낮다.

2. Scaling — $\div \sqrt{d_k}$

d_k 가 커질수록 dot product의 크기도 커진다. 큰 값이 softmax에 들어가면

$$[2, 3, 50] \xrightarrow{\text{softmax}} [0, 0, 1]$$

처럼 분포가 극단화되어 gradient가 매우 작아질 수 있다.

3. Softmax

score를 attention weight로 바꿔, 각 token을 얼마나 참고할지 결정한다.

4. Weighted Sum of Values

attention weight × Value. 관련성이 높은 Value를 더 많이 반영한다.

⇒ **Attention** = 관련성을 계산해 필요한 정보를 **weighted sum**하는 과정

보충: 왜 하필 $\sqrt{d_k}$ 인가

논문 각주의 근거를 따라가면, 임의의 상수가 아니라 **분산을 1로 맞추는 정규화**임을 알 수 있다.

q, k 의 각 성분이 서로 독립이고 평균 0, 분산 1을 따른다고 두면

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i \quad \Rightarrow \quad \mathbb{E}[q \cdot k] = 0, \quad \text{Var}[q \cdot k] = d_k$$

표준편차가 $\sqrt{d_k}$ 이므로, $\sqrt{d_k}$ 로 나누면 분산이 1이 된다. 즉 $\sqrt{\cdot}$ 는 dot product의 표준편차이며, 값의 스케일을 차원과 무관하게 만든다.

한편 6.2절 ablation에서는 d_k 를 줄이면 성능이 떨어진다고 보고한다. Query-Key compatibility를 계산할 만큼의 representation capacity가 필요하다는 뜻이다. 즉 d_k 는 scaling과 표현력 양쪽에 동시에 걸린 값이다.

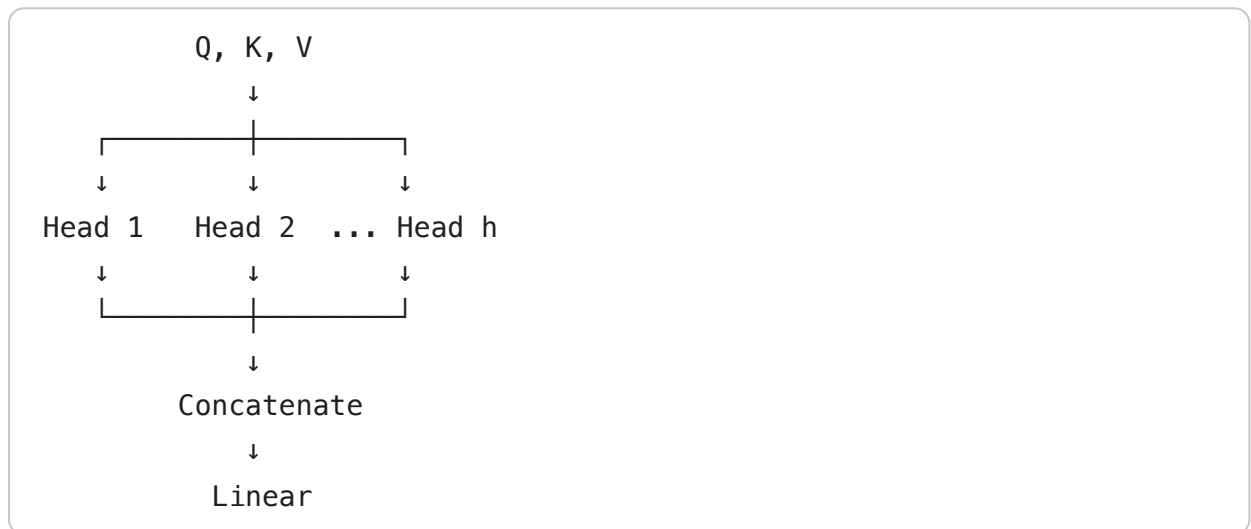
3.2.2 Multi-Head Attention

문제 — single attention은 하나의 representation space에서만 관계를 계산한다. 하지만 문장 안에는 의미적 관계, 문법적 관계, 대명사-reference, long-range dependency 등 여러 관계가 겹쳐 있다.

해결 — 여러 attention head를 병렬로 사용하고, head마다 서로 다른 linear projection을 적용한다.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$



논문 설정: $h = 8, d_{\text{model}} = 512, d_k = d_v = 64$

각 head의 차원을 줄여 전체 computational cost는 full-dimensional single-head와 비슷하게 유지한다.

보충: 텐서 shape 추적

문장 길이를 n 이라 할 때, head 하나를 끝까지 따라가면 다음과 같다.

단계	연산	Shape	메모
입력	X	$(n, 512)$	
projection	$Q = XW^Q$	$(n, 64)$	head 하나 기준, $d_k = 64$
score	QK^T	(n, n)	여기가 quadratic cost 의 정체
가중합	$\text{softmax}(\cdot)V$	$(n, 64)$	head 하나의 출력
결합	8 heads concat	$(n, 512)$	$8 \times 64 = 512$

단계	연산	Shape	메모
출력	$\cdot W^O$	$(n, 512)$	입력과 같은 차원 → residual 덧셈 가능

$h \times d_k = d_{\text{model}}$ 은 우연이 아니라 **residual connection**이 강제하는 제약의 결과다.

⇒ **Multi-Head Attention** = 동일한 **sequence**를 여러 관점에서 동시에 보고 결과를 결합

3.2.3 Applications of Attention in Transformer

연산 방식은 동일하지만 **Q/K/V**의 출처가 다르다.

유형	Q	K/V	역할
Encoder Self-Attention	Encoder	Encoder	입력 내부 관계
Decoder Masked Self-Attention	Decoder	Decoder	이전 출력 관계 (미래 위치 mask)
Encoder-Decoder Attention	Decoder	Encoder	입력-출력 연결

⇒ 연산은 같고, **Q/K/V**가 어디에서 오는지가 다름

3.3 Position-wise Feed-Forward Networks

Attention으로 token 간 정보를 교환한 뒤, 각 token의 representation을 추가로 변환한다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

512 → Linear → 2048 → ReLU → Linear → 512

position-wise: 각 position에 동일한 FFN을 독립적으로 적용

Token 1 → FFN → Token 1'
 Token 2 → FFN → Token 2'
 Token 3 → FFN → Token 3'

	역할
Attention	token ↔ token 정보 교환
FFN	각 token 내부 representation 변환

⇒ Attention으로 정보를 모으고, FFN으로 가공

보충: 파라미터는 실제로 어디에 있는가

Encoder layer 1개 기준으로 직접 계산해 보면,

구성	계산	파라미터 수
Attention	4	$(W^Q,)$
	$\times 512$	$W^K,$
	$\times 512$	$W^V,$
		W^O
		약 1.05M

구성	계산	파라미터 수
FFN	512	약 2.10M
	× 2048	
	+ 2048	
	× 512	

"Transformer = attention 모델"이라 부르지만, **파라미터의 약 2/3는 FFN에 있다**. 3.3절이 겠다리가 아닌 이유이며, 이후 MoE 계열이 왜 FFN을 표적으로 삼는지도 여기서 설명된다.

3.4 Embeddings and Softmax

Input — token을 learned embedding으로 변환, 차원은 $d_{\text{model}} = 512$

Output

Decoder Output → Linear → Softmax → Next-token Probability

Weight Sharing — input embedding, output embedding, pre-softmax linear transformation이 동일한 weight matrix를 공유한다. embedding에서는 weight에 $\sqrt{d_{\text{model}}}$ 을 곱한다.

3.5 Positional Encoding

문제 — Transformer는 recurrence도 convolution도 없이 모든 token을 동시에 처리한다. 따라서 architecture 자체에는 순서 정보가 없다.

RNN : Token 1 → Token 2 → Token 3 (계산 순서 = 위치 정보)
 Transformer : Token 1
 Token 2 → 동시에 계산
 Token 3

해결

Input = Token Embedding + Positional Encoding

즉 "무슨 token인가"에 "몇 번째 token인가"를 더한다.

Sinusoidal Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- pos : token 위치
- i : embedding dimension
- dimension마다 서로 다른 frequency 사용

Learned Positional Embedding과 비교

learned 방식도 실험했고 성능은 거의 동일했다. 그럼에도 sinusoidal을 택한 이유는

- relative position 학습이 쉬울 것이라는 가정
- training보다 긴 sequence로 extrapolation 가능성

⇒ 병렬화를 위해 사라진 **sequence order**를 **Positional Encoding**으로 보충

4. Why Self-Attention

세 가지 기준으로 비교한다: computational complexity, sequential operations, maximum path length.

Layer	Complexity	Sequential Ops	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Restricted Self-Attention*	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

* 논문이 향후 방향으로 제안만 하고 실험하지 않은 변형

4.1 Computational Complexity

Self-Attention은 $O(n^2 d)$, Recurrent는 $O(nd^2)$ 이다. machine translation에서는 일반적으로 $n < d$ 인 경우가 많다.

⇒ 이 조건에서 Self-Attention이 recurrent layer보다 계산 효율적이다. 즉 우위는 조건부다.

4.2 Parallelization

- RNN: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots \rightarrow x_n$, sequential operations $O(n)$
- Self-Attention: 모든 token 관계를 matrix operation으로 동시에 계산, sequential operations $O(1)$

⇒ GPU parallelization에 유리

4.3 Maximum Path Length

maximum path length: 두 위치의 정보를 연결할 때 지나야 하는 최대 연산 단계 수

RNN은 $O(n)$, CNN은 $O(\log_k n)$, Self-Attention은 $O(1)$ 이다.

4.4 Long-Range Dependency

두 위치 사이 path가 짧을수록 long-range dependency 학습이 쉽다. RNN/CNN은 먼 token의 정보를 여러 단계를 거쳐 전달하지만, Self-Attention은 한 번의 attention으로 직접 연결한다.

4.5 Interpretability

attention weight는 현재 token이 어떤 다른 token을 얼마나 참고했는지를 나타낸다.

visualization에서 서로 다른 head가 long-distance dependency, anaphora, sentence structure 등과 관련된 pattern을 학습하는 모습이 관찰된다.

4.6 Self-Attention의 한계

모든 token pair를 계산하므로 $O(n^2d)$ 이며, sequence length n 이 커지면 비용이 급격히 증가한다. 논문은 주변 r 개 token만 보는 **Restricted Self-Attention**을 제안한다.

⇒ sequential computation 문제는 해결했지만, 긴 **sequence의 quadratic cost**가 새로운 문제로 남았다

5. Training

5.1 Training Data and Batching

	English-German	English-French
Dataset	WMT 2014	WMT 2014
Sentence pairs	약 4.5M	약 36M
Vocabulary	BPE, shared 약 37K	word-piece 약 32K

Batching — 비슷한 sequence length끼리 batch를 구성하며, batch당 source/target token은 각각 약 25K.

5.2 Hardware and Schedule

Hardware: **8 × NVIDIA P100 GPUs**

Model	Steps	Time/Step	Total
Base	100K	약 0.4 sec	약 12시간
Big	300K	약 1 sec	약 3.5일

5.3 Optimizer

Adam, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step^{-0.5}, step \cdot warmup_steps^{-1.5})$$

$warmup_steps = 4000$

1. 초기 4,000 step: learning rate 선형 증가
2. 이후: inverse square root에 따라 감소

Learning Rate

/\

/ _____
4000

참고: 이 warmup이 사실상 필수였던 이유는 원 논문이 Post-LN(sub-layer 뒤에 LayerNorm) 구조를 썼기 때문이다. LayerNorm을 sub-layer 앞으로 옮긴 Pre-LN 구조에서는 warmup 의존이 크게 줄어든다. 5.3절의 스케줄과 3.1절의 구조는 사실 같은 이야기였던 셈이다.

5.4 Regularization

Residual Dropout — sub-layer output과 embedding + positional encoding에 적용.

Base model에서 $P_{drop} = 0.1$

Label Smoothing — $\epsilon_{ls} = 0.1$. 정답 token에 확률 1을 완전히 부여하지 않아 model의 과도한 confidence를 줄인다. perplexity는 악화되지만 accuracy와 BLEU는 개선된다.

6. Results

6.1 Machine Translation

Task	Model	BLEU	비고
EN-DE	Transformer Big	28.4	기존 최고 model/ensemble 대비 2 BLEU 이상 개선
EN-FR	Transformer Big	41.8	높은 single-model 성능

Training Cost

Model	EN-DE BLEU	Training Cost (FLOPs)
Transformer Base	27.3	3.3×10^{18}
Transformer Big	28.4	2.3×10^{19}

⇒ 기존 경쟁 모델보다 낮은 **training cost**로 높은 성능

수치를 읽을 때의 조건: BLEU 28.4는 마지막 20개 checkpoint를 평균하고, beam size 4, length penalty $\alpha = 0.6$ 으로 디코딩한 값이다. 단일 모델이지만 checkpoint averaging은 사용했다는 점이 앙상블과 비교할 때 중요하다.

6.2 Model Variations

변경 요소	결과
Attention head 수	single-head는 성능 감소, multi-head는 개선, 과도하면 다시 감소
Key dimension d_k	줄이면 성능 감소 → 충분한 representation capacity 필요
Model size (d_{model}, d_{ff})	키우면 전반적으로 성능 향상
Dropout	없으면 overfitting 및 성능 저하
Positional Encoding	sinusoidal vs learned 거의 동일

⇒ positional encoding은 방식보다 **position** 정보를 제공하는 것 자체가 중요

6.3 English Constituency Parsing

translation 외 task로의 일반화를 확인하기 위해 Penn Treebank WSJ를 사용했다.

Setting	Transformer F1
WSJ only	91.3
Semi-supervised	92.7

task-specific tuning을 많이 하지 않았음에도 기존 model과 경쟁력 있는 성능을 보였다.

⇒ **Transformer는 translation 전용이 아닌 general sequence architecture**

7. Conclusion

RNN

- sequence를 잘 처리
- 그러나 sequential computation
- 병렬화 어려움

↓

RNN/CNN 제거 → Self-Attention

- 각 token이 필요한 정보를 직접 탐색

결과

- parallelization ↑, maximum path length ↓
- long-range dependency 학습 용이
- training speed ↑, Machine Translation SOTA
- 다른 sequence task에도 generalization

Future Work — image, audio, video, 긴 sequence, generation의 sequential computation 감소

⇒ **sequence 정보를 한 token에서 다음 token으로 순차 전달하는 대신, 각 token이 필요한 정보를 직접 가져오는 구조로 전환**

보충: 2017년의 선택 중 무엇이 살아남았나

구성 요소	원 논문 (2017)	현재 표준	바뀐 이유
Normalization	Post-LN	Pre-LN · RMSNorm	초기 학습 불안정 완화 → warmup 의존 감소
위치 정보	Sinusoidal PE	RoPE · ALiBi	상대 위치를 직접 다루고 길이 외삽에 유리
FFN 활성화	ReLU	SwiGLU 계열	같은 파라미터 예산에서 더 나은 성능
Attention 형태	MHA ($h = 8$)	GQA · MQA	추론 시 KV 캐시 메모리 절감
긴 sequence 대응	Restricted attention 제한	FlashAttention	근사 대신 IO 최적화로 정확도 손실 없이 해결

Appendix. Attention Visualizations

Figure 3. Long-Distance Dependency — encoder self-attention layer 5에서, `making`이 멀리 떨어진 `more difficult` 관련 표현에 attention한다.

⇒ 먼 token 간 dependency를 직접 연결

Figure 4. Anaphora Resolution — `its`에 대해 특정 head가 관련 표현에 강한 attention을 보인다.

⇒ 일부 head가 대명사-reference 관계와 유사한 역할을 학습

Figure 5. Different Attention Heads — 서로 다른 head가 서로 다른 sentence structure 관련 pattern을 보인다.

⇒ Multi-Head Attention의 head들이 서로 다른 역할을 학습할 수 있음